POLARIS
SCHOOL OF TECHNOLOGY

GENERATIVE AI - CONSOLIDATED STUDY NOTES

From Text to MiniGPT

Tokenization, BPE, statistical embeddings, attention, Transformer architecture, and a minimal GPT implementation in PyTorch.



Core idea. A language model never directly receives meaning. It receives token IDs, converts them to learned vectors, repeatedly mixes and transforms those vectors, and finally produces a probability distribution over the next token.

Part	Focus	Outcome
I	Text processing and BPE	Understand how text becomes a finite sequence of token IDs.
II	Statistical intuition for embeddings	Understand why learned vectors capture similarity and latent structure.
III	Attention and Transformers	Understand Q/K/V, self-attention, cross-attention, FFN, normalization, residuals, and depth.
IV	MiniGPT practical	Implement and train a tiny decoder-only Transformer in Python.

Designed as lecture notes and revision material for a Generative AI course. Equations are intentionally light: enough mathematics to explain the mechanism without turning the notes into a proof-oriented treatment.

CONTENTS

Study Guide Map

The document is organized from representation to modeling, so each section builds on the previous one.

Study Guide Map 2

Text Processing and Byte Pair Encoding 5

1. Why text must be tokenized . . . 5

2. Byte Pair Encoding: the training idea . . . 5

3. Worked tokenization example 6

4. Special tokens . . . 6

5. From token IDs to tensors . . . 6

6. Why token count matters computationally 7

7. Padding and masks . . . 7

8. Part I checkpoint . . . 7

Why Embeddings Work: A Statistical View 8

9. The problem with discrete IDs . . . 8

10. Distributional statistics: context is evidence 9

11. Co-occurrence matrix . . . 9

12. PMI: separating association from raw frequency 10

13. Low-dimensional latent factors . . . 10

14. Cosine similarity and vector geometry 11

15. Why "meaning" appears without labels . . . 11

16. Embeddings in a modern Transformer 12

17. How training moves an embedding . . . 12

18. Part II checkpoint . . . 12

Attention and Transformer Models 13

19. Why attention was needed . . . 13

20. Q, K and V intuition . . . 13

21. Self-attention: one numerical example 14

22. The full scaled dot-product formula . . . 14

23. Attention matrices and tensor shapes 15

24. Why multi-head attention? . . . 15

25. Causal masking **16**

26. Self-attention vs cross-attention 16

27. Why positional information is necessary **17**

28. Embedding + position 17

29. Why the feed-forward network exists **18**

30. Why nonlinearity matters 18

31. Residual connections **19**

32. Why LayerNorm / RMSNorm is used 19

33. Anatomy of a decoder Transformer block **20**

34. Encoder, decoder and encoder-decoder 20

35. From hidden states to vocabulary logits **21**

36. Cross-entropy loss 21

37. Weight tying 21

38. How much attention should we stack? **22**

39. Why not 1,000 layers? 22

40. Training vs inference **23**

41. KV cache 23

42. Sampling 23

43. Attention complexity and efficient variants **24**

44. Common misconceptions 24

45. Complete Transformer mental model **25**

46. Shape walkthrough 25

Build a MiniGPT in Python **26**

47. Classroom design 26

48. Dataset 26

49. Tokenizer and batches **27**

50. Causal self-attention module **28**

51. Transformer block **29**

52. MiniGPT model 29

53. Forward pass and loss **30**

54. Training loop **31**

55. What backpropagation is learning 31

56. Autoregressive generation **32**

57. Experiments for students 32

58. Debugging checklist **33**

59. Mapping MiniGPT to a production LLM 33

Subjective Questions **34**

60. One-page revision sheet **35**

Selected foundational reading **36**

Recommended teaching order. Teach Part I before embeddings, then use Part II to explain why vectors can represent statistics. Part III becomes much easier once students are comfortable with vectors and matrix shapes. Finish with Part IV so they can connect every concept to executable code.

PART I

Text Processing and Byte Pair Encoding

Why tokenization exists, how BPE learns a vocabulary, and how token IDs become model input.

1. Why text must be tokenized

Neural networks operate on numbers with fixed tensor shapes. Raw text is variable-length, symbolic, multilingual, and contains effectively unlimited possible strings. A tokenizer creates a finite vocabulary of reusable pieces and converts text into integer IDs.

- Word-level tokenization is simple but produces a huge vocabulary and an out-of-vocabulary problem.
- Character-level tokenization avoids unknown words but makes sequences much longer.
- Subword tokenization gives a useful compromise: common patterns become single tokens while rare words can still be decomposed.

> Mental model. The tokenizer is a deterministic preprocessing system. The language model does not learn the BPE merge operations during normal model training; it learns how to use the resulting token sequence.

2. Byte Pair Encoding: the training idea

BPE starts with small units and repeatedly merges the most frequent adjacent pair. Each merge creates a new vocabulary item. Common strings become compact tokens; uncommon strings remain decomposable.

Step	Current representation	Frequent pair	New token
0	l o w | l o w e r | l o w e s t	l + o	lo
1	lo w | lo w e r | lo w e s t	lo + w	low
2	low | low e r | low e s t	e + r (example)	er

The exact sequence of merges depends on frequencies in the full training corpus. The important distinction is that BPE training learns merge rules; later, BPE tokenization applies those learned rules to new text.

3. Worked tokenization example

Suppose the learned vocabulary contains the pieces low, er, and est. Then:

Input	Possible BPE tokens	Token IDs (illustrative)
lower	[low, er]	[314, 52]
lowest	[low, est]	[314, 91]
slowest	[s, low, est]	[19, 314, 91]

A vocabulary table maps every token to a unique integer ID. The numeric magnitude of an ID has no semantic meaning: token 314 is not "more" or "closer" to token 52 than token 91.

4. Special tokens

Token type	Purpose
BOS / start	Marks the beginning of a sequence in some models.
EOS / end	Marks the end of generated or training text.
PAD	Fills unused positions when examples are batched to a common length.
UNK	Represents unknown symbols in tokenizers that permit unknowns.

Important. Modern byte-level or byte-fallback tokenizers can usually represent arbitrary text without a traditional unknown-word failure.

5. From token IDs to tensors

For a batch of B sequences, each with T tokens, the tokenizer typically produces an integer tensor with shape:

```
[B, T]
```

If one sentence becomes 28 tokens and the model dimension is 512, the embedding lookup will produce a matrix of shape:

```
[28, 512] for one sentence or [B, 28, 512] for a batch
```

6. Why token count matters computationally

In ordinary dense self-attention, the model compares positions with positions. The attention score matrix for a sequence of length T has approximately T x T entries per head.

$$\text{attention score count} \propto T^2$$

This is why tokenization quality affects more than vocabulary convenience. If the same paragraph becomes 500 tokens under one tokenizer and 700 under another, attention on the longer representation requires roughly:

$$(700 / 500)^2 = 1.96$$

or almost twice as many pairwise attention scores, ignoring other costs.

7. Padding and masks

Batches often contain sequences with different lengths. Padding creates a rectangular tensor, while an attention mask tells the model which positions are real and which are padding.

Example	Tokens after padding	Mask
A	[12, 41, 9, PAD, PAD]	[1, 1, 1, 0, 0]
B	[8, 7, 5, 4, 2]	[1, 1, 1, 1, 1]

Causal mask. Decoder-only language models also hide future positions. Token i may attend to positions 0...i but not to later tokens. This enforces next-token prediction.

8. Part I checkpoint

- BPE compresses frequent character/subword patterns into reusable vocabulary pieces.
- Token IDs are labels, not semantic coordinates.
- Sequence length T directly affects memory and attention cost.
- The output of tokenization is the discrete input to the embedding layer.

PART II

Why Embeddings Work: A Statistical View

From co-occurrence counts to learned vector geometry, with just enough mathematics to see why meaning-like structure emerges.

9. The problem with discrete IDs

Representation	cat	dog	What it means
Integer ID	421	912	Only labels; distance between IDs is meaningless.
One-hot	[1,0,0,...]	[0,1,0,...]	Different tokens are equally orthogonal.
Embedding	[0.7,0.2,...]	[0.68,0.24,...]	Coordinates can encode learned statistical similarity.

An embedding is a row in a trainable matrix E. If the vocabulary contains V tokens and the model dimension is d, then:

```
E ∈ R^(V x d)
```

Embedding matrix E

-0.6	-0.4	-0.2	0.0	0.2
-0.2	0.2	0.6	1.0	-0.8
0.2	0.8	-0.8	-0.2	0.4
0.6	-0.8	0.0	0.8	-0.6

← lookup row for token ID

Looking up token ID i simply selects row E[i]. During training, gradients adjust these rows so that they become useful for prediction.

10. Distributional statistics: context is evidence

A token is observed through the contexts in which it appears. If two words tend to occur around similar neighbors, they often play related linguistic roles. This is the distributional intuition behind classical and neural embeddings.

Target token	Example nearby words
cat	pet, fur, animal, sleeps, kitten
dog	pet, fur, animal, barks, puppy
car	road, engine, drive, wheel, vehicle

From a corpus we can estimate conditional probabilities such as:

```
P(context = "pet" | target = "cat")
```

If the context distributions for cat and dog are similar, a low-dimensional model has an incentive to represent them with similar coordinates.

11. Co-occurrence matrix

Imagine a matrix X where rows are target words and columns are contexts. Entry X(i,j) counts how often target i appears near context j.

	pet	fur	road	engine
cat	42	31	0	0
dog	47	28	1	0
car	0	0	39	44

Statistical interpretation. Rows that look similar represent tokens that behave similarly in the corpus. Embeddings compress these high-dimensional behavior profiles into a smaller set of latent coordinates.

12. PMI: separating association from raw frequency

Raw counts can be dominated by very frequent words. Pointwise Mutual Information asks whether two events occur together more often than chance would predict.

```
PMI(w,c) = log( P(w,c) / (P(w) P(c)) )
```

Interpretation:

- PMI > 0: the word and context co-occur more than expected under independence.
- PMI ≈ 0: their co-occurrence is close to chance.
- PMI < 0: they co-occur less than expected.

Suppose P(cat)=0.01, P(pet)=0.02, and P(cat,pet)=0.001. Then:

```
PMI(cat,pet) = log(0.001 / (0.01 x 0.02)) = log(5)
```

The pair is positively associated because it occurs about five times more often than independent probabilities would suggest.

13. Low-dimensional latent factors

The full co-occurrence matrix may have one dimension per context word, which is enormous. A factorization approach approximates it using lower-dimensional matrices:

$$X \approx W C^{\top}$$

Rows of W can be used as embeddings. Neural methods such as Word2Vec arrive at a similar result through prediction: parameters that help predict context become the vector representation.

14. Cosine similarity and vector geometry

Once tokens are vectors, we can compare their directions. Cosine similarity is commonly used because it measures angle rather than raw magnitude.

```
cos(a,b) = (a · b) / (||a|| ||b||)
```

If a = [1,1] and b = [0.9,1.1], their dot product is 2.0. The two vectors point in almost the same direction, so cosine similarity is close to 1.

Cosine value	Interpretation
Near +1	Vectors point in similar directions.
Near 0	Vectors are roughly orthogonal.
Near -1	Vectors point in opposing directions.

Do not over-interpret axes. An embedding is not naturally a 2D or 3D point. A model might use 256, 768, 4096, or more dimensions. 2D/3D plots are projections used only for visualization.

15. Why "meaning" appears without labels

Training does not usually contain a rule saying cat and dog must be close. Instead, the loss function rewards parameters that improve prediction. Because cat and dog appear in statistically similar contexts, using related internal features is efficient. Semantic-looking geometry emerges as a side effect of solving many prediction problems.

16. Embeddings in a modern Transformer

A Transformer normally begins with a learned token embedding table. The initial vector is context-free: every occurrence of the same token starts from the same lookup row. After attention layers, each occurrence becomes a different contextual hidden state.

Stage	"bank" in river bank	"bank" in bank loan
Token embedding	Same initial row E[bank]	Same initial row E[bank]
After attention	Influenced by river / water context	Influenced by loan / finance context
Final hidden state	Contextual representation A	Contextual representation B

17. How training moves an embedding

A simplified next-token model computes logits from a hidden vector h and output weights, then applies softmax. Cross-entropy produces a gradient. Backpropagation sends part of that gradient into the embedding rows that participated in the prediction.

```
p_i = exp(z_i) / Σ_j exp(z_j)
```

```
L = -log p_target
```

Repeated over billions of examples, the embedding matrix becomes a compact statistical memory of how tokens participate in prediction.

> Key distinction. Token embeddings are parameters. Contextual hidden states are computed activations. They live in the same model-dimensional space, but they are not the same object.

18. Part II checkpoint

- Embeddings compress high-dimensional context statistics into trainable coordinates.
- Similarity is learned because similar vectors help solve similar prediction tasks.
- There is no special "3D semantic space" inside the model; the true space is d_model-dimensional.
- Contextual meaning is produced by the Transformer layers, not by the embedding lookup alone.

PART III

Attention and Transformer Models

Why attention exists, how self-attention and cross-attention work, and why a Transformer block contains position, residuals, normalization and an FFN.

19. Why attention was needed

Sequence models must allow information from one token to affect another. Recurrent networks pass information step by step, which creates a long path between distant positions. Attention creates a direct content-dependent interaction between positions.

Question	Without attention	With attention
How does token 100 use token 3?	Information must propagate through many recurrent steps.	Token 100 can assign weight directly to token 3.
Can interactions be parallelized?	Limited by sequential recurrence.	All query-key comparisons can be computed as matrices.
Is every prior token equally useful?	State compression must preserve what matters.	Weights can focus on the most relevant positions.

Core idea. Attention is a learned routing mechanism: each token asks what information it needs, finds positions that advertise relevant information, then blends their content.

20. Q, K and V intuition

Object	Layman interpretation	Computed from
Query Q	What am I looking for?	Current token representation x W_Q
Key K	What kind of information do I contain?	Token representation x W_K
Value V	What information should I contribute?	Token representation x W_V

Each input vector is projected three times using learned matrices. Q and K determine where to look; V determines what to retrieve.

21. Self-attention: one numerical example

Consider the sentence "The cat sleeps". Suppose the query for cat and the three keys are:

```
Q_cat = [1, 1]
```

```
K_The = [1,0], K_cat = [1,1], K_sleeps = [0,1]
```

Ignoring scaling for a moment, raw dot-product scores are:

```
[1,1]·[1,0] = 1 | [1,1]·[1,1] = 2 | [1,1]·[0,1] = 1
```

so the score vector is [1, 2, 1]. Softmax converts these arbitrary real scores into positive weights that sum to 1:

```
softmax([1,2,1]) ≈ [0.212, 0.576, 0.212]
```

If the values are V_The, V_cat and V_sleeps, the new representation for cat is approximately:

```
0.212 V_The + 0.576 V_cat + 0.212 V_sleeps
```

> What changed? The cat vector is no longer only the embedding of "cat". It now contains a weighted mixture of information from the entire visible context.

22. The full scaled dot-product formula

$$\text{Attention}(Q,K,V) = \text{softmax}\left(QK^{\top} / \sqrt{d_k} \right) V$$

The division by √d_k keeps dot products from growing too large as the key/query dimension increases. Without scaling, softmax can become extremely peaked early in training, which reduces useful gradient signal.

23. Attention matrices and tensor shapes

For batch size B, sequence length T, model dimension d_model, H heads, and head dimension d_head = d_model/H:

Tensor	Typical shape
Input hidden states X	[B, T, d_model]
Q, K, V after splitting heads	[B, H, T, d_head]
Attention scores QK^T	[B, H, T, T]
Weighted values	[B, H, T, d_head]
Concatenated heads	[B, T, d_model]

Example: d_model = 256 and H = 8 gives:

```
d_head = 256 / 8 = 32
```

Each head has its own learned projections. The common phrase "split the embedding into heads" is a useful shape intuition, but implementations usually perform learned projections and then reshape the projected representation.

24. Why multi-head attention?

A single attention distribution must express one pattern of relevance at a time. Multiple heads give the layer multiple learned subspaces and multiple attention maps. Different heads may specialize in short-range syntax, entity reference, delimiter structure, positional patterns, or other useful relationships.

> Important nuance. We should not assume a head has a clean human-readable role. Head specialization is learned, distributed, and often redundant.

25. Causal masking

A decoder language model must predict the next token using only earlier tokens. For T=4, a causal visibility matrix looks like:

```
position 0: [ visible, hidden,  hidden,  hidden ]
position 1: [ visible, visible, hidden,  hidden ]
position 2: [ visible, visible, visible, hidden ]
position 3: [ visible, visible, visible, visible ]
```

Implementation typically adds a very negative number to forbidden logits before softmax, causing their probabilities to become approximately zero.

26. Self-attention vs cross-attention

Type	Queries come from	Keys/values come from	Use case
Self-attention	Same sequence	Same sequence	Tokens communicate within one representation.
Cross-attention	Decoder/current sequence	Another encoded sequence	One sequence retrieves information from another.

In an encoder-decoder Transformer for translation, the encoder first produces source representations. The decoder performs causal self-attention over generated target tokens and then cross-attends to the encoder output.

In a decoder-only GPT, ordinary text generation uses causal self-attention without a separate encoder cross-attention stage.

27. Why positional information is necessary

Pure self-attention is permutation-equivariant: without a position signal, it does not inherently know which token came first. The set of token vectors for "dog bites man" and "man bites dog" contains the same token identities, even though the meaning is different.

The model therefore needs a representation of order. Common approaches include:

- Learned absolute positional embeddings: add a learned vector for position 0, 1, 2, ...
- Sinusoidal positions: use deterministic sine/cosine functions at multiple frequencies.
- RoPE (Rotary Position Embedding): rotate query/key components as a function of position so relative offsets affect attention scores.
- Relative position biases: directly bias attention based on distance or relative position.

> RoPE intuition. Position is injected into the geometry used by Q and K. Two tokens with the same content at different positions receive different rotations, so their dot product can depend on relative distance.

28. Embedding + position

With simple learned positions:

```
x_t = token_embedding[token_t] + position_embedding[t]
```

The result still has shape [B, T, d_model]. Position does not require adding a new tensor dimension; it modifies the representation already used by the Transformer.

29. Why the feed-forward network exists

Attention mixes information across tokens. After that communication step, each token needs a powerful nonlinear transformation of its own features. That is the role of the position-wise feed-forward network (FFN).

$$FFN(x) = W_2\,\sigma(W_1 x + b_1) + b_2$$

The same FFN weights are applied independently to every sequence position.

Component	Primary job
Attention	Move / mix information between token positions.
FFN	Transform features within each token representation.

Many modern FFNs expand the dimension, apply an activation such as GELU or SwiGLU, then project back down. For example, d_model=768 might use an internal dimension of roughly 3072 in a classic 4x expansion design.

> Mental model. Attention = communication. FFN = computation. A Transformer block needs both.

30. Why nonlinearity matters

If every layer were only linear matrix multiplication, stacking them would collapse into one larger linear transformation. Nonlinear activations allow the network to build piecewise and compositional feature transformations that cannot be represented by a single matrix.

31. Residual connections

A residual connection adds the block input back to the transformed output:

$$y = x + F(x)$$

This creates a direct path through the network. A layer can learn a small correction rather than reconstructing the entire representation from scratch, and gradients have shorter routes during backpropagation.

32. Why LayerNorm / RMSNorm is used

Deep networks repeatedly change activation distributions. Normalization keeps representation scales controlled and improves optimization stability. LayerNorm normalizes features within each token vector:

$$LN(x) = \gamma \left(\frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}\right) + \beta$$

where μ and σ^2 are computed across the feature dimension of that token. RMSNorm is a related alternative that normalizes using root-mean-square magnitude without subtracting the mean.

Mechanism	Why it helps
Residual connection	Preserves information and improves gradient flow.
LayerNorm / RMSNorm	Controls scale and stabilizes optimization.
Dropout (when used)	Regularizes by randomly masking activations during training.

Pre-norm vs post-norm. Modern decoder-only LLMs often normalize before attention/FFN sublayers because pre-norm architectures are easier to optimize at large depth. Exact details vary across model families.

33. Anatomy of a decoder Transformer block



One common pre-norm block can be written as:

```
a = x + Attention(Norm(x))
y = a + FFN(Norm(a))
```

Stack L copies of this block. The output of one block becomes the input to the next. Earlier layers often represent local/lexical patterns; deeper layers have had more opportunities for multi-step contextual computation.

34. Encoder, decoder and encoder-decoder

Architecture	Attention visibility	Typical tasks
Encoder-only	Bidirectional self-attention	Classification, retrieval embeddings, masked-token objectives.
Decoder-only	Causal self-attention	Autoregressive text/code generation.
Encoder-decoder	Encoder self-attention + decoder causal self-attention + cross-attention	Translation, conditional sequence generation.

35. From hidden states to vocabulary logits

After the final Transformer block, each position has a contextual vector h_t with dimension d_model. A linear output layer maps it to one score per vocabulary item:

```
logits_t = h_t W_vocab + b
```

If vocabulary size V=50,000, then each position produces a vector of 50,000 logits. Softmax turns these into next-token probabilities.

For a batch:

```
[B, T, d_model] -> output projection -> [B, T, V]
```

During training, the targets are shifted by one token. The model at position t is trained to predict token t+1.

36. Cross-entropy loss

For a single target token y:

```
L = -log P(y | previous tokens)
```

The total loss is usually averaged across many token positions and batch examples. Backpropagation updates all parameters that contributed to those probabilities: embeddings, attention projections, FFNs, normalization parameters, and output weights.

37. Weight tying

Some language models reuse the token embedding matrix as the output vocabulary projection (or a transpose-equivalent form). This is called weight tying. It reduces parameters and links input/output token geometry, although implementation details vary.

38. How much attention should we stack?

There is no formula that says a task requires exactly N Transformer layers. Depth, width, number of heads, context length and dataset size interact. Model design is normally chosen through compute constraints, scaling studies and validation performance.

Choice	What increasing it tends to provide	Cost / risk
Depth L	More sequential stages of reasoning / feature refinement	More latency, memory, optimization difficulty.
Width d_model	Richer feature space per token	Parameters and matrix-multiply cost grow quickly.
Heads H	More parallel attention subspaces	Too many tiny heads can reduce per-head capacity.
Context T	Access to more tokens	Dense attention memory/compute grows $\sim T^2$.

A useful design constraint is:

```
d_head = d_model / H
```

Common models often keep d_head in a moderate range and scale overall capacity by changing d_model and layer count. There is no universal best number of heads.

39. Why not 1,000 layers?

- Training cost and inference latency increase with depth.
- Optimization becomes harder even with residuals and normalization.
- The dataset may not justify the extra capacity, leading to diminishing returns or overfitting.
- Hardware memory and communication become limiting factors.
- A wider or better-trained model may produce more benefit than simply making it deeper.

> Practical selection. For a class project, choose the smallest architecture that clearly demonstrates learning. For research or production, compare validation loss / downstream quality under a fixed compute budget.

40. Training vs inference

Phase	What happens
Training	Process many tokens in parallel, compute next-token loss at every visible position, backpropagate, update weights.
Prefill	At generation time, process the prompt and populate attention key/value states.
Decode	Generate one token at a time, append it, and reuse cached K/V states for earlier tokens.

41. KV cache

During autoregressive decoding, previous tokens do not change. Recomputing all of their keys and values at every step would waste work. The KV cache stores them so the next token only needs to compute its new Q/K/V and attend to the cached history.

This improves generation speed but consumes memory approximately proportional to:

```
layers x sequence_length x heads x head_dimension
```

for both keys and values, multiplied by batch size and numerical precision.

42. Sampling

Softmax gives probabilities, but generation still needs a selection rule. Greedy decoding chooses the highest-probability token. Temperature rescales logits; top-k and top-p restrict the candidate set. These change generation behavior without retraining the Transformer.

43. Attention complexity and efficient variants

Dense self-attention stores or computes a T x T score structure per head, giving $O(T^2)$ pairwise interactions. Long-context systems therefore use techniques such as:

- FlashAttention: computes exact attention with hardware-aware tiling to reduce memory traffic.
- Sliding-window / local attention: restricts each token to a neighborhood.
- Sparse attention: evaluates only selected token pairs.
- Grouped-query / multi-query attention: reduces the number of distinct K/V heads to shrink KV-cache cost.
- Linear / kernel approximations: restructure or approximate attention to avoid explicit quadratic score matrices.

44. Common misconceptions

Misconception	Correction
"Attention chooses one word."	Softmax usually creates a weighted mixture of many visible values.
"An embedding is meaning stored in 3D."	It is a learned vector in d_model dimensions; meaning-like structure is distributed.
"More heads always means a better model."	Heads trade off with head dimension and total compute.
"The FFN is only an output layer."	Every Transformer block normally contains an FFN; the vocabulary output projection is separate.
"Causal mask deletes future tokens."	It prevents attention to them for a given query position; the tokens may still exist elsewhere in the training tensor.

45. Complete Transformer mental model



Inside each of the L blocks:



A useful one-sentence summary is:

> Transformer block. Attention lets tokens communicate; the FFN lets each token compute; residuals preserve a stable information path; normalization keeps the deep stack trainable.

46. Shape walkthrough

Step	Shape for B=4, T=64, d_model=256, H=8, V=10,000
Token IDs	[4, 64]
Embedding output	[4, 64, 256]
Q/K/V per head	[4, 8, 64, 32]
Attention scores	[4, 8, 64, 64]
Block output	[4, 64, 256]
Vocabulary logits	[4, 64, 10,000]

Students who can explain every shape in this table usually understand the mechanical core of a Transformer.

PART IV

Build a MiniGPT in Python

A minimal decoder-only Transformer that can be trained on a tiny character-level corpus in well under 10 minutes on a normal CPU.

47. Classroom design

Setting	Value
Tokenizer	Character-level, for teaching simplicity
Vocabulary	Characters present in the tiny corpus
Context length	64 characters
Embedding size	64
Attention heads	4
Transformer blocks	2
Training	~600 AdamW updates, batch size 32
Objective	Predict the next character

Why character-level? It removes BPE implementation complexity from the coding lab. The model still demonstrates embeddings, causal self-attention, FFNs, residuals, LayerNorm, cross-entropy and autoregressive generation.

48. Dataset

Use a small repetitive corpus so the model can learn visible patterns quickly. For example, repeat short lines such as:

```
the cat sits on the mat.
the dog sits by the door.
the bird sings in the tree.
the cat sleeps by the fire.
the dog runs in the yard.
```

The goal is not language quality. The goal is to make the loss fall and generation become more corpus-like within a classroom session.

49. Tokenizer and batches

```
text = open("tiny_corpus.txt", "r", encoding="utf-8").read()
chars = sorted(set(text))
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}

def encode(s):
    return [stoi[c] for c in s]

def decode(ids):
    return "".join(itos[i] for i in ids)

data = torch.tensor(encode(text), dtype=torch.long)
```

A training example uses a window of context. If x contains characters 0...63, the target y is the same window shifted by one position.

```
x = data[i : i + block_size]
y = data[i + 1 : i + block_size + 1]
```

Therefore x and y both have shape [T]. After stacking B random windows, they have shape [B,T].

Next-token objective. At every position, y contains the character that immediately follows the prefix visible at that position. One training window therefore supplies T supervised prediction targets.

50. Causal self-attention module

```
class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads, block_size):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads = n_heads
        self.d_head = d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.proj = nn.Linear(d_model, d_model)
        self.register_buffer(
            "mask",
            torch.tril(torch.ones(block_size, block_size))
        )

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        q = q.view(B,T,self.n_heads,self.d_head).transpose(1,2)
        k = k.view(B,T,self.n_heads,self.d_head).transpose(1,2)
        v = v.view(B,T,self.n_heads,self.d_head).transpose(1,2)

        scores = q @ k.transpose(-2,-1) / math.sqrt(self.d_head)
        scores = scores.masked_fill(self.mask[:T,:T] == 0, float("-inf"))
        weights = torch.softmax(scores, dim=-1)
        out = weights @ v
        out = out.transpose(1,2).contiguous().view(B,T,C)
        return self.proj(out)
```

This code is the mathematical attention formula translated almost directly into tensor operations.

51. Transformer block

```
class Block(nn.Module):
    def __init__(self, d_model, n_heads, block_size):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = CausalSelfAttention(d_model, n_heads, block_size)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, 4 * d_model),
            nn.GELU(),
            nn.Linear(4 * d_model, d_model),
        )

    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        x = x + self.ffn(self.ln2(x))
        return x
```

Notice the exact conceptual structure from Part III: pre-normalization, attention, residual, normalization, FFN, residual.

52. MiniGPT model

```
class MiniGPT(nn.Module):
    def __init__(self, vocab_size, block_size=64, d_model=64,
                 n_heads=4, n_layers=2):
        super().__init__()
        self.block_size = block_size
        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(block_size, d_model)
        self.blocks = nn.ModuleList([
            Block(d_model, n_heads, block_size)
            for _ in range(n_layers)
        ])
        self.ln_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size)
```

53. Forward pass and loss

```
def forward(self, idx, targets=None):
    B, T = idx.shape
    pos = torch.arange(T, device=idx.device)
    x = self.token_emb(idx) + self.pos_emb(pos)[None, :, :]

    for block in self.blocks:
        x = block(x)

    logits = self.head(self.ln_f(x))

    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.view(B*T, -1),
            targets.view(B*T)
        )
    return logits, loss
```

Shape walkthrough:

Line	Shape
idx	[B,T]
token_emb(idx)	[B,T,64]
x after blocks	[B,T,64]
logits	[B,T,V]
flattened logits for loss	[B*T,V]
flattened targets	[B*T]

Why flatten for cross-entropy? PyTorch treats each token position as one classification example over V classes. Flattening combines batch and time into a single example dimension.

54. Training loop

```
torch.manual_seed(42)
model = MiniGPT(vocab_size=len(chars))
opt = torch.optim.AdamW(model.parameters(), lr=3e-3)

for step in range(600):
    xb, yb = get_batch()
    logits, loss = model(xb, yb)

    opt.zero_grad(set_to_none=True)
    loss.backward()
    opt.step()

    if step % 100 == 0:
        print(step, float(loss))
```

The expected behavior is not a specific loss value, because hardware, PyTorch version and corpus details can change results. What matters is a clear downward trend and increasingly structured generated text.

55. What backpropagation is learning

- Token embeddings move so characters that participate in similar predictions can reuse features.
- Q/K projections learn which positions should interact.
- V projections learn what information should be passed when a position is attended to.
- FFN weights learn nonlinear feature transformations.
- Output weights learn how hidden features map to next-character logits.

Classroom measurement. Print parameter count and training time. A ~100K-parameter model on a tiny corpus should train comfortably within a 10-minute lab; on many CPUs it will be much faster.

56. Autoregressive generation

```
@torch.no_grad()
def generate(model, idx, max_new_tokens=100):
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -model.block_size:]
        logits, _ = model(idx_cond)
        logits = logits[:, -1, :]
        probs = torch.softmax(logits, dim=-1)
        next_id = torch.multinomial(probs, num_samples=1)
        idx = torch.cat([idx, next_id], dim=1)
    return idx
```

Generation repeatedly runs the same next-token model. The newly sampled token becomes part of the next input context. This is why GPT is called autoregressive.

57. Experiments for students

Experiment	Prediction
Remove causal mask	Training loss may look easier, but the model can cheat by seeing future tokens.
Set n_layers from 2 to 1	Fewer transformation stages; may train faster but underfit patterns.
Increase d_model 64 -> 128	More parameters and feature capacity; slower training.
Set n_heads 4 -> 8 at same d_model	Each head becomes smaller (16 -> 8 dimensions).
Remove positional embeddings	Order-dependent patterns become much harder to learn.

58. Debugging checklist

Symptom	Likely check
Loss is NaN	Learning rate, invalid masking, infinities, bad tensor dtype.
Model sees future tokens	Verify lower-triangular causal mask and score masking before softmax.
Shape error in attention	Check d_model % n_heads == 0 and transpose/view order.
Generation repeats one symbol	Training data too small, learning rate, sampling temperature, or under/over-training.
Loss never decreases	Targets not shifted, optimizer not stepping, parameters detached, or dataset too random.

59. Mapping MiniGPT to a production LLM

MiniGPT	Production-scale model
Character vocabulary	BPE / unigram / byte-aware subword tokenizer
d_model = 64	Hundreds to many thousands of hidden dimensions
2 layers	Dozens or more layers
64-token context	Thousands to very long-context windows
Tiny corpus	Massive curated text/code/multimodal datasets
Simple MHA	Optimized attention, GQA/MQA, RoPE, FlashAttention, KV cache

Architectural continuity. The scale changes dramatically, but the classroom model still contains the recognizable mechanism: token embeddings, position, causal attention, FFN, residuals, normalization, logits, next-token loss, and autoregressive generation.

ASSESSMENT

Subjective Questions

Three concept-heavy questions with light mathematics, suitable for written assessment or viva preparation.

Question 1 - BPE and tokenization

Consider the corpus low lower lowest. Assume BPE initially treats every character as a separate symbol. (a) Explain why subword tokenization is preferred over a purely word-level vocabulary. (b) Identify a frequent pair and show two possible merge steps. (c) If the final vocabulary contains low, er, and est, show how lower and lowest are tokenized. (d) A sentence becomes 28 tokens and d_model=512. What is the embedding shape for one sentence, and why does token count influence Transformer compute?

> Expected mathematical point: embedding shape = [28,512]; dense attention score work grows approximately with T^2.

Question 2 - Self-attention

For the sentence "The cat sleeps", let Q_cat=[1,1], K_The=[1,0], K_cat=[1,1], and K_sleeps=[0,1]. (a) Explain Q, K and V intuitively. (b) Compute the raw dot-product scores for cat. (c) Which token receives the largest score? (d) Explain what softmax does and why the resulting weights are applied to V vectors. (e) Why is this more expressive than processing each token independently?

> Expected mathematical point: raw scores = [1,2,1]; softmax produces normalized positive weights, approximately [0.212,0.576,0.212].

Question 3 - Transformer architecture

A decoder-only Transformer has vocabulary size 10,000, sequence length 50, d_model=256 and 8 attention heads. (a) Give the embedding output shape for one sentence. (b) Calculate d_head. (c) Explain why positional information is necessary using "dog bites man" and "man bites dog". (d) Explain the roles of self-attention, FFN, residual connection and LayerNorm. (e) What does increasing from 2 to 12 blocks potentially provide, and why would simply using 1,000 blocks be a poor default design strategy?

> Expected mathematical point: embedding output = [50,256]; d_head = 256/8 = 32.

60. One-page revision sheet

Concept	Remember this
BPE	Repeatedly merge frequent adjacent symbols; apply learned merges to new text.
Token ID	A discrete label, not a semantic number.
Embedding	Trainable row E[token_id] in a d_model-dimensional space.
Position	Needed because attention alone does not encode sequence order.
Q / K / V	Question / advertisement / content.
Self-attention	softmax($QK^T/\sqrt{d_k}$)V using one sequence as Q,K,V source.
Cross-attention	Q from one sequence; K,V from another representation.
Multi-head	Multiple learned attention subspaces in parallel.
FFN	Nonlinear per-token feature transformation after communication.
Residual	x + F(x): stable information and gradient path.
LayerNorm	Controls per-token feature scale; stabilizes deep training.
Causal mask	Prevents a decoder position from using future tokens.
Logits	Unnormalized vocabulary scores.
Softmax	Turns logits into a probability distribution.
Cross-entropy	Penalizes low probability assigned to the correct next token.
KV cache	Reuses old keys/values during autoregressive decoding.

End-to-end sentence. Text is tokenized into IDs; IDs are embedded; position is injected; repeated Transformer blocks use attention to communicate and FFNs to compute; the final hidden state is projected to vocabulary logits; softmax defines next-token probabilities; training uses cross-entropy and backpropagation; inference samples a token and repeats.

REFERENCES

Selected foundational reading

These references are useful for deeper study beyond the lecture-note level.

- Vaswani, A. et al. (2017). Attention Is All You Need. Introduced the Transformer architecture based entirely on attention.
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. Popularized BPE-style subword modeling for neural machine translation.
- Mikolov, T. et al. (2013). Efficient Estimation of Word Representations in Vector Space. Introduced influential Word2Vec methods.
- Radford, A. et al. (2018-2019). GPT and GPT-2 technical reports. Demonstrated decoder-only pretraining at increasing scale.
- Brown, T. et al. (2020). Language Models are Few-Shot Learners. GPT-3 scaling and in-context learning.
- Su, J. et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. Introduced RoPE.
- Dao, T. et al. (2022). FlashAttention. Hardware-aware exact attention with reduced memory traffic.

Submission note. This document is structured as study notes: concise derivations, worked examples, implementation excerpts, and revision prompts. For a formal research report, add course-specific citation style, institution-required metadata, and any instructor-mandated bibliography format.